APCS 程式設計系列 · U05 · 實作中級 · C++ / Python 對照版

二維陣列與矩陣模擬 重點講義

宣告與走訪（列 / 行）· 初始化陷阱 · 轉置與旋轉 90 度 · 方向陣列與邊界檢查 · 地圖模擬 · 螺旋走訪

1. 二維陣列：先列（橫的）、再行（直的）

二維陣列就是「表格」。**a[i][j]** 的 i 是第幾**列**（橫排，往下數），j 是第幾**行**（直排，往右數），兩個都從 0 開始。n 列 m 行的表格，i 從 0 到 n-1、j 從 0 到 m-1。

C++ 讀入 n 列 m 行的矩陣，印出每一列的和

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> a(n, vector<int>(m));
    for (int i = 0; i < n; i++)      // i：第幾列
        for (int j = 0; j < m; j++)  // j：第幾行
            cin >> a[i][j];
    for (int i = 0; i < n; i++) {
        int s = 0;
        for (int j = 0; j < m; j++) s += a[i][j];
        cout << s << "\n";
    }
}
```

Python

```python
n, m = map(int, input().split())
a = [list(map(int, input().split())) for _ in range(n)]
for i in range(n):          # i：第幾列
    s = 0
    for j in range(m):      # j：第幾行
        s += a[i][j]
    print(s)
```

輸入	輸出
2 3 1 2 3 4 5 6	6 15

走訪順序決定輸出順序

- **列優先**：外層跑 i、內層跑 j，一列一列由左到右（最常見，也是記憶體裡的存放順序）。
- **行優先**：外層跑 j、內層跑 i，一行一行由上到下。
- 列優先攤平成一維：a[i][j] 是第 i × m + j 個；反過來，第 k 個是 a[k / m][k % m]。

2. 建立表格：初始化與「共用同一列」陷阱

寫法	語言	說明
int a[105][105];（寫在 main 外面）	C / C++	全域陣列自動全部是 0；大小要先開好
vector<vector<int>> a(n, vector<int>(m, 0));	C++	n 列 m 行、全部是 0，大小可以用變數
a = [[0] * m for _ in range(n)]	Python	每一列都是新建立的串列（正確）
a = [[0] * m] * n	Python	n 列全是「同一個」串列（錯誤）

C++ 建立 3 列 2 行、全部是 0 的表格

```cpp
int n = 3, m = 2;
vector<vector<int>> g(n, vector<int>(m, 0));
g[0][0] = 5;
for (int i = 0; i < n; i++)
    cout << g[i][0] << " " << g[i][1] << "\n";
```

Python

```python
n, m = 3, 2
g = [[0] * m for _ in range(n)]   # 每一列都是新建立的
g[0][0] = 5
for row in g:
    print(row[0], row[1])
```

輸入	輸出
（不需要輸入）	5 0 0 0 0 0

Python [[0] * m] * n 的陷阱：改一格，整行都變

```python
g = [[0] * 2] * 3     # 錯誤：3 列其實是同一個串列
g[0][0] = 5
print(g)
```

輸入	輸出
（不需要輸入）	[[5, 0], [5, 0], [5, 0]]

* 3 只是把「同一列的參考」複製 3 份，所以改 g[0] 就等於改到每一列。二維串列一律用 for 迴圈（串列生成式）建立。

陷阱提醒

【錯】C++ 區域陣列沒初始化是垃圾值；Python 的 [[0] * m] * n 每一列是同一個串列。

【對】C 的 {1, 2, 3, 4, 5} 依列優先填入、其餘補 0；Python 用 [[0] * m for _ in range(n)]。

- 陣列開「n 列 m 行」還是「m 列 n 行」，先看題目的輸入格式，寫錯就會越界或讀錯。

3. 轉置與旋轉 90 度

C++ 順時針旋轉 90 度

```cpp
int n = 2, m = 3;
int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
int b[3][2];          // 轉完是 m 列 n 行
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        b[j][n - 1 - i] = a[i][j];
for (int i = 0; i < m; i++)
    cout << b[i][0] << " " << b[i][1] << "\n";
```

Python

```python
a = [[1, 2, 3], [4, 5, 6]]
n, m = len(a), len(a[0])
b = [[0] * n for _ in range(m)]  # m 列 n 行
for i in range(n):
    for j in range(m):
        b[j][n - 1 - i] = a[i][j]
for row in b:
    print(*row)
```

輸入	輸出
（不需要輸入）	4 1 5 2 6 3

操作	新陣列大小	公式（原本 a 是 n 列 m 行）
轉置	m 列 n 行	b[j][i] = a[i][j]
順時針 90 度	m 列 n 行	b[j][n-1-i] = a[i][j]
逆時針 90 度	m 列 n 行	b[m-1-j][i] = a[i][j]
旋轉 180 度	n 列 m 行	b[n-1-i][m-1-j] = a[i][j]
左右翻轉	n 列 m 行	b[i][m-1-j] = a[i][j]

公式記不住？用一個角驗證

- 順時針轉：原本「左下角」a[n-1][0] 會跑到新陣列的「左上角」b[0][0]。把 i = n-1、j = 0 代進公式檢查一次就不會錯。
- 順時針 90 度 = 先上下翻轉，再轉置。逆時針 = 先轉置，再上下翻轉。
- Python 速寫：轉置 zip(*a)；順時針 zip(*a[::-1])（每一列再轉成 list）。

陷阱提醒

【錯】旋轉後忘了 n、m 對調：新陣列是 m 列 n 行，連續旋轉好幾次時每次都要換。

【錯】原地轉置兩層都從 0 跑到 n-1：每一對 (i, j) 會被交換兩次，又換回原樣。

【對】原地轉置（只限正方形）內層從 j = i + 1 開始，只換對角線右上半部。

4. 方向陣列 dr / dc 與邊界檢查

地圖題最常要看「上下左右」的鄰居。把四個方向的位移存成兩個陣列，用一個迴圈跑完，就不用寫四段幾乎一樣的程式。**先算出新位置、檢查沒有出界，才去讀那一格。**

C++ 方向陣列：印出上下左右的鄰居（出界就跳過）

```cpp
int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
int n = 3, m = 3, r = 0, c = 1;
int dr[4] = {-1, 1, 0, 0};       // 上、下、左、右
int dc[4] = {0, 0, -1, 1};
for (int k = 0; k < 4; k++) {
    int nr = r + dr[k], nc = c + dc[k];
    if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
    cout << a[nr][nc] << " ";
}
cout << "\n";
```

Python

```python
a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
n, m, r, c = 3, 3, 0, 1
dr = [-1, 1, 0, 0]               # 上、下、左、右
dc = [0, 0, -1, 1]
for k in range(4):
    nr, nc = r + dr[k], c + dc[k]
    if nr < 0 or nr >= n or nc < 0 or nc >= m:
        continue
    print(a[nr][nc], end=" ")
print()
```

輸入	輸出
（不需要輸入）	`5 1 3`

四方向與八方向

- 四方向（上下左右）：dr = {-1, 1, 0, 0}、dc = {0, 0, -1, 1}，兩個陣列「同一個位置」配成一對。
- 八方向（含斜角）：dr、dc 各從 -1 跑到 1 的兩層迴圈，記得跳過 (0, 0) 自己；或直接寫 8 個元素的陣列。
- 要「轉彎」的題目，方向照順時針排（右、下、左、上）：右轉 d = (d + 1) % 4，左轉 d = (d + 3) % 4，迴轉 d = (d + 2) % 4。

陷阱提醒 邊界檢查的三個坑

【錯】先讀 a[nr][nc] 再檢查有沒有出界：順序反了，檢查就沒有意義。

【錯】越界不一定會報錯：C/C++ 會默默讀到別的值；Python 的 a[-1] 會讀到最後一列。

【對】邊界檢查寫在 && / and 的前面，靠短路求值保護；或在地圖外圍多開一圈「牆」，就不用檢查邊界。

5. 地圖模擬：數周圍、走格子

C++ 踩地雷：數出每一格周圍 8 格的地雷數

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    for (auto &row : g) cin >> row;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (g[i][j] == '*') { cout << '*'; continue; }
            int cnt = 0;
            for (int r = i - 1; r <= i + 1; r++)
                for (int c = j - 1; c <= j + 1; c++)
                    if (r >= 0 && r < n && c >= 0 && c < m
                        && g[r][c] == '*') cnt++;
            cout << cnt;
        }
        cout << "\n";
    }
}
```

Python

```python
n, m = map(int, input().split())
g = [input() for _ in range(n)]
for i in range(n):
    line = ""
    for j in range(m):
        if g[i][j] == "*":
            line += "*"
            continue
        cnt = 0
        for r in range(i - 1, i + 2):
            for c in range(j - 1, j + 2):
                if 0 <= r < n and 0 <= c < m and g[r][c] == "*":
                    cnt += 1
        line += str(cnt)
    print(line)
```

輸入	輸出
3 4 *..* .... .*..	*11* 2221 1*10

走格子模擬的固定步驟

- 位置用 (r, c) 兩個變數記；每個指令先換成方向編號 k，再查 dr[k]、dc[k]。
- 先算「下一格」(nr, nc)，確認合法（沒出界、不是牆）才真的移動；不合法就留在原地。
- 要數「走過幾個不同的格子」：開一個同樣大小的 visited 表格，第一次踩到才加 1。
- 每一步只做 O(1) 的事，指令有 k 個就是 O(k)，十萬個指令也很快。

6. 螺旋走訪

C++ 螺旋走訪：碰到邊界或走過的格子就右轉

```cpp
int n = 3, m = 4, a[3][4], v[3][4] = {};
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++) a[i][j] = i * m + j + 1;
int dr[4] = {0, 1, 0, -1}, dc[4] = {1, 0, -1, 0};  // 右下左上
int r = 0, c = 0, d = 0;
for (int k = 0; k < n * m; k++) {
    cout << a[r][c] << " ";
    v[r][c] = 1;
    int nr = r + dr[d], nc = c + dc[d];
    if (nr < 0 || nr >= n || nc < 0 || nc >= m || v[nr][nc]) {
        d = (d + 1) % 4;                        // 右轉
        nr = r + dr[d]; nc = c + dc[d];
    }
    r = nr; c = nc;
}
cout << "\n";
```

Python

```python
n, m = 3, 4
a = [[i * m + j + 1 for j in range(m)] for i in range(n)]
v = [[False] * m for _ in range(n)]
dr, dc = [0, 1, 0, -1], [1, 0, -1, 0]   # 右下左上
r = c = d = 0
out = []
for _ in range(n * m):
    out.append(a[r][c])
    v[r][c] = True
    nr, nc = r + dr[d], c + dc[d]
    if not (0 <= nr < n and 0 <= nc < m) or v[nr][nc]:
        d = (d + 1) % 4                    # 右轉
        nr, nc = r + dr[d], c + dc[d]
    r, c = nr, nc
print(*out)
```

輸入	輸出
（不需要輸入）	`1 2 3 4 8 12 11 10 9 5 6 7`

- 方向照「右、下、左、上」排好，碰到邊界或走過的格子就 d = (d + 1) % 4 右轉。
- 總共走 n × m 步，每一格剛好走一次：O(nm)。
- 另一種寫法：記住上、下、左、右四條邊界，每走完一條邊就把那條邊界往內縮一格，直到邊界交錯。
- 螺旋「填數」也一樣：把「印出 a[r][c]」改成「a[r][c] = 第幾步」，用 a[r][c] 不是 0 代表走過。

讀完這份，你應該要會

- □ 看到 a[i][j] 知道 i 是列（橫）、j 是行（直），會寫列優先和行優先的走訪
- □ C++ 會開 vector<vector<int>>；Python 會用 for 建表、不用 [[0] * m] * n
- □ 會寫順時針旋轉 b[j][n-1-i] = a[i][j]，並記得新陣列是 m 列 n 行
- □ 會用 dr / dc 方向陣列跑四方向、八方向，而且邊界檢查寫在讀取之前
- □ 會用 d = (d + 1) % 4 右轉，寫出螺旋走訪

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室

IG @jf_chuang

LINE 諮詢 jenfoo2021